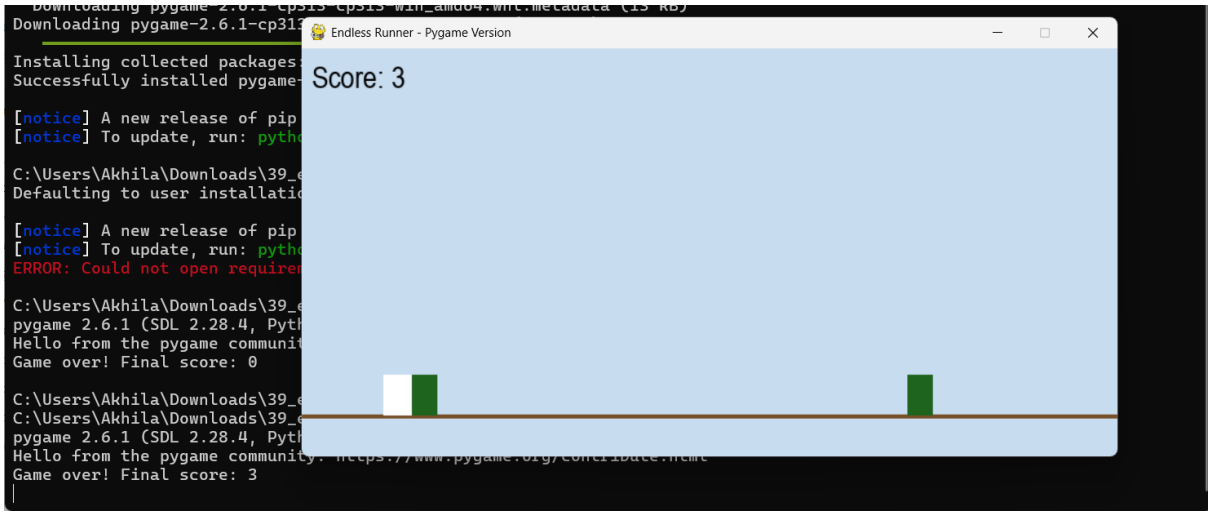
SE LAB – 4

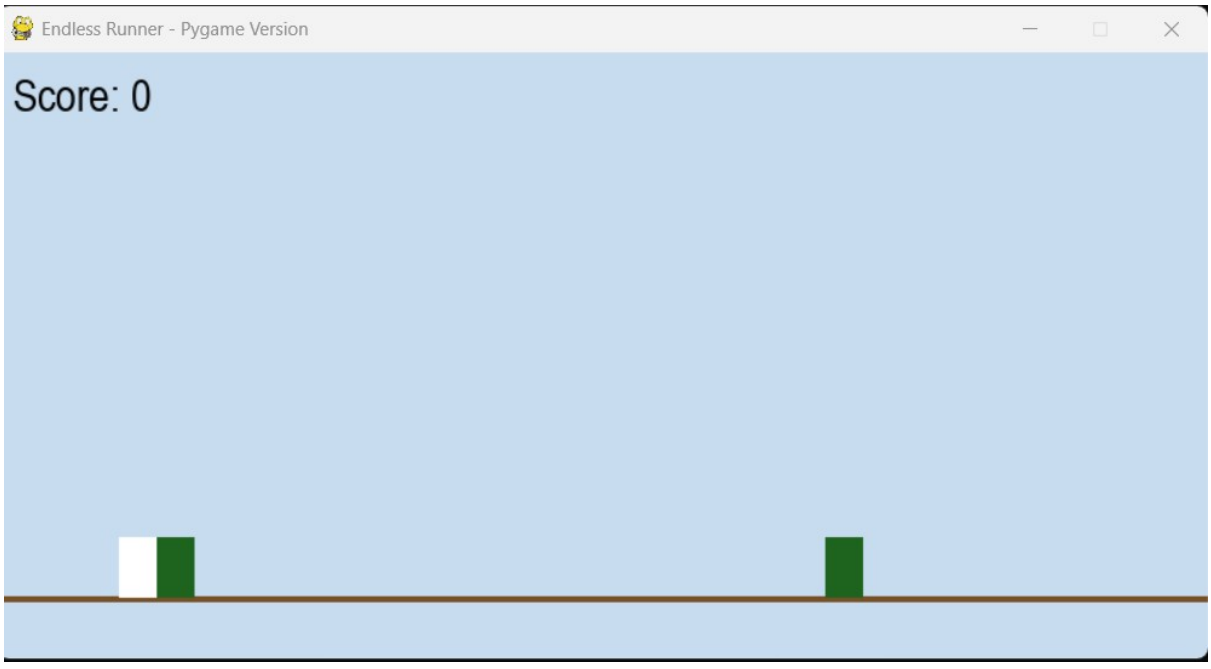
NAME: AKHILA T

SRN: PES2UG24CS043

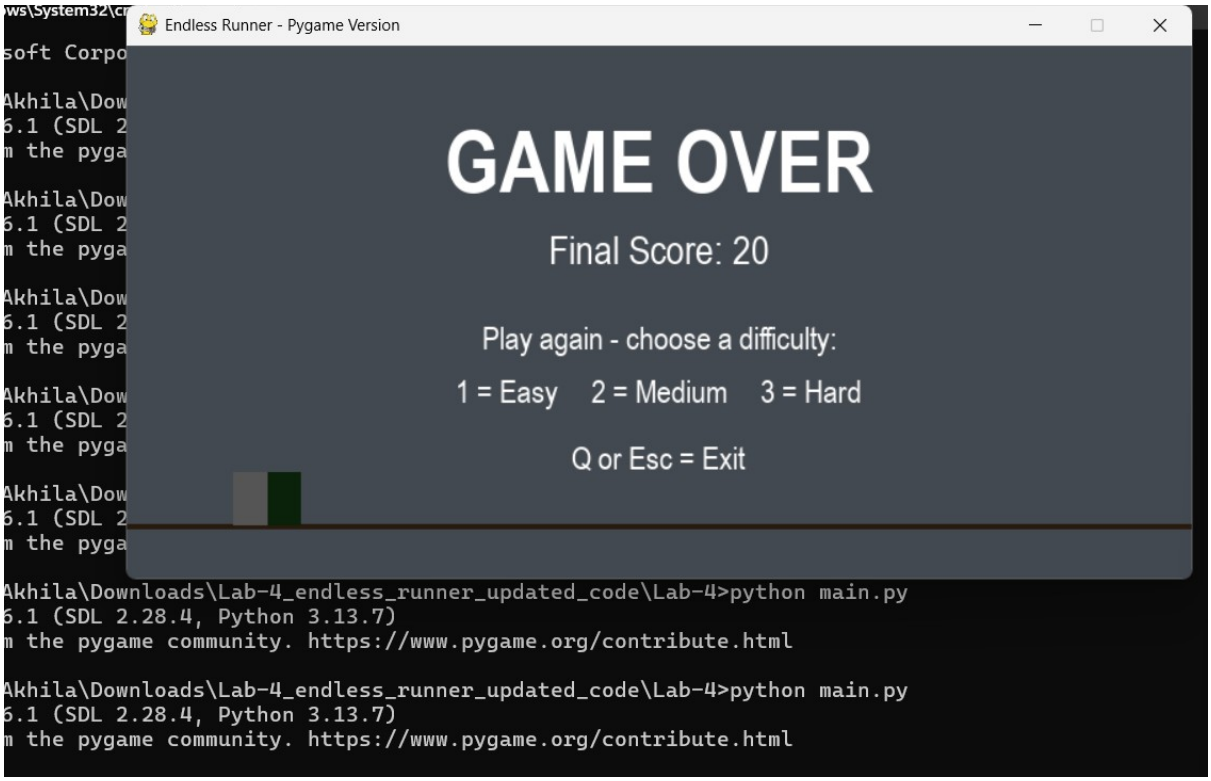
Topic: 39_endless-runners

BEFORE UPDATION:



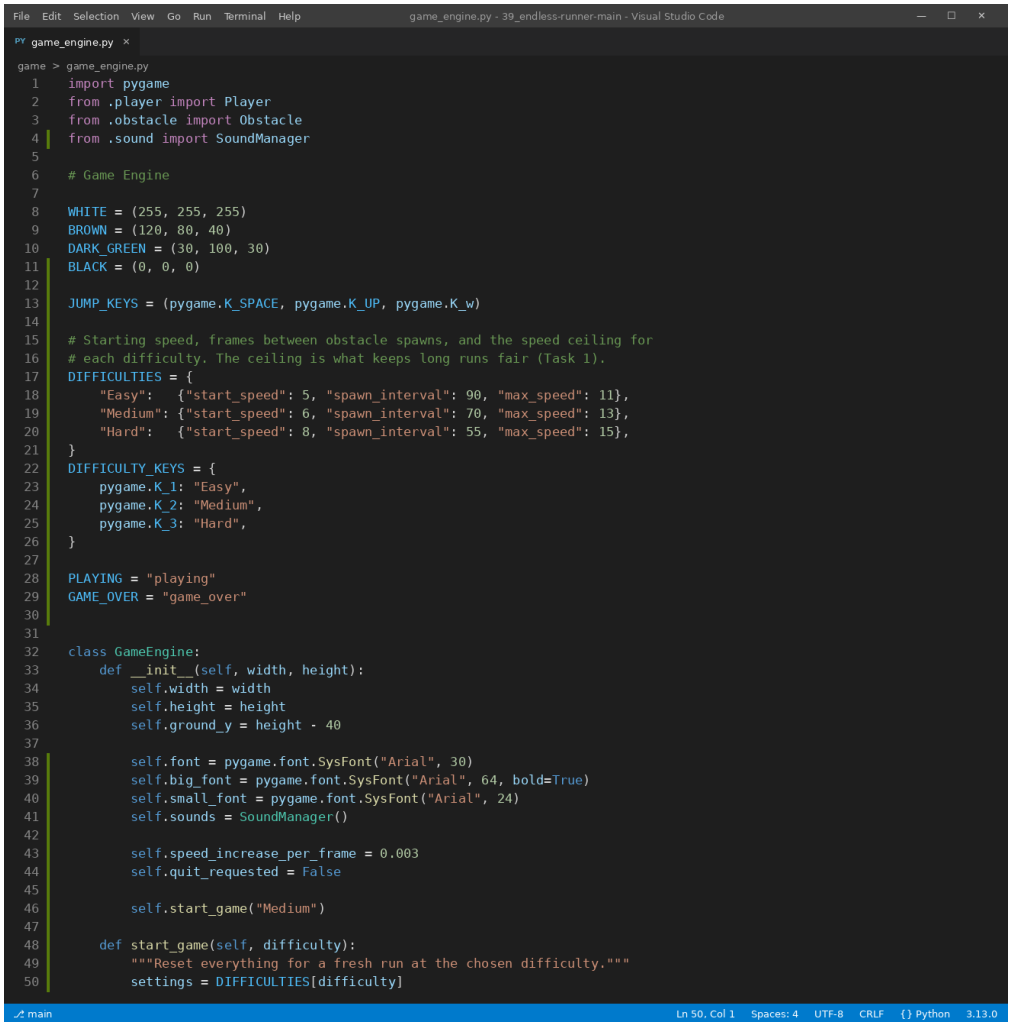


AFTER UPDATION:

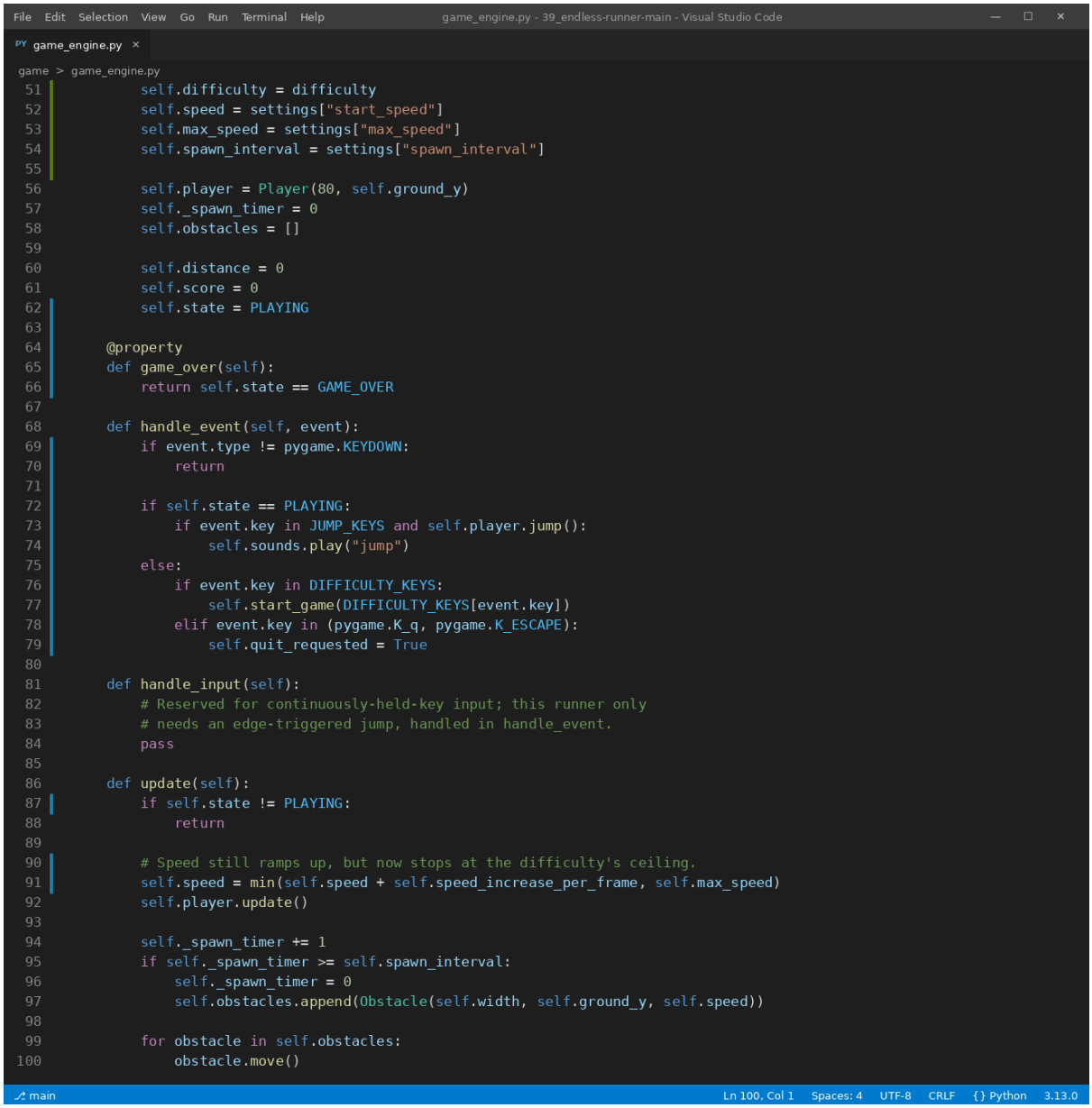


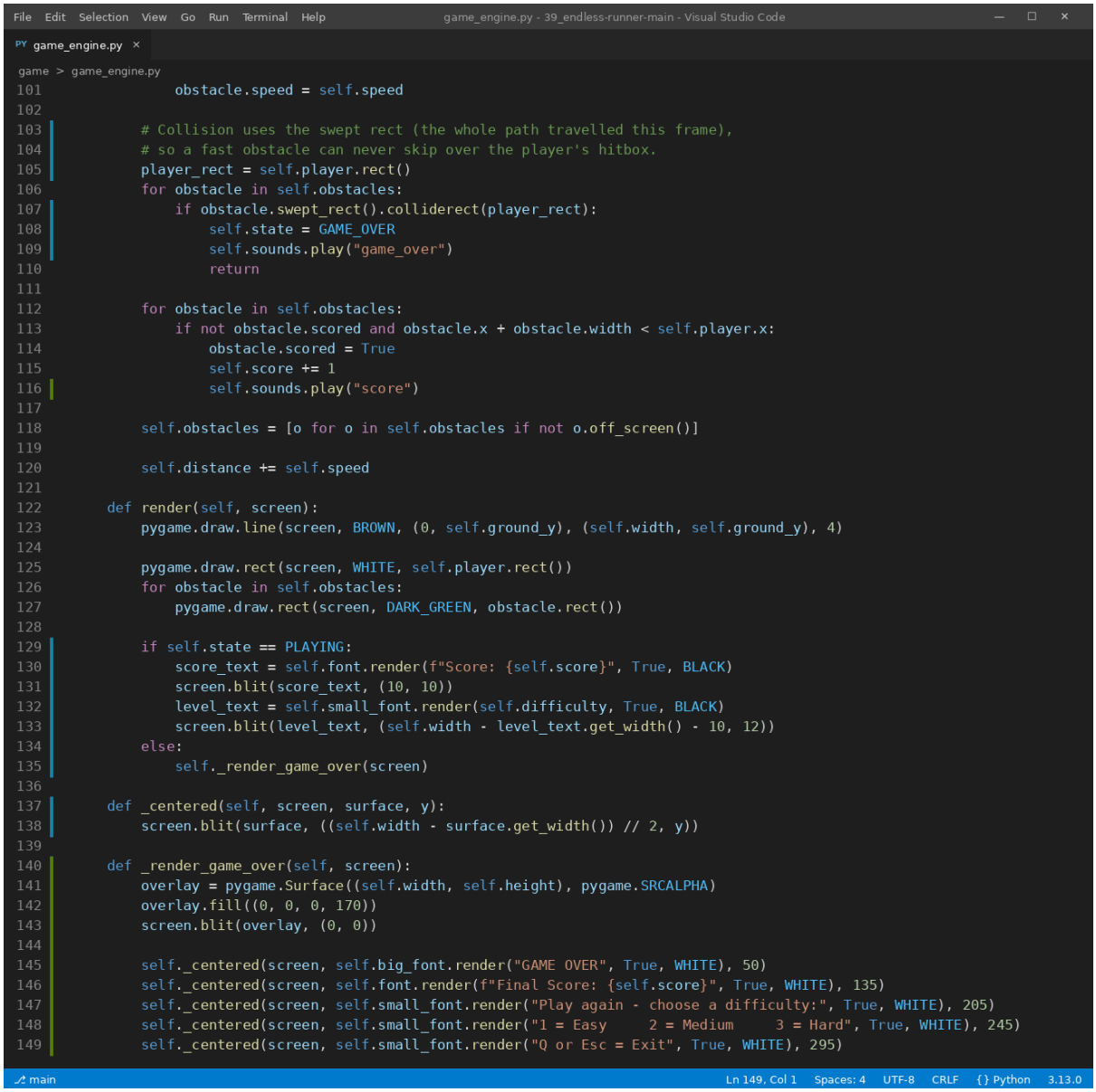
CODE CHANGED:

game/game_engine.py (speed)



game/obstacle.py


File Edit Selection View Go Run Terminal Help
game_engine.py - 39_endless-runner-main - Visual Studio Code
game_engine.py
game > game_engine.py
51 self.difficulty = difficulty
52 self.speed = settings["start_speed"]
53 self.max_speed = settings["max_speed"]
54 self.spawn_interval = settings["spawn_interval"]
55
56 self.player = Player(80, self.ground_y)
57 self._spawn_timer = 0
58 self.obstacles = []
59
60 self.distance = 0
61 self.score = 0
62 self.state = PLAYING
63
64 @property
65 def game_over(self):
66 return self.state == GAME_OVER
67
68 def handle_event(self, event):
69 if event.type != pygame.KEYDOWN:
70 return
71
72 if self.state == PLAYING:
73 if event.key in JUMP_KEYS and self.player.jump():
74 self.sounds.play("jump")
75 else:
76 if event.key in DIFFICULTY_KEYS:
77 self.start_game(DIFFICULTY_KEYS[event.key])
78 elif event.key in (pygame.K_q, pygame.K_ESCAPE):
79 self.quit_requested = True
80
81 def handle_input(self):
82 # Reserved for continuously-held-key input; this runner only
83 # needs an edge-triggered jump, handled in handle_event.
84 pass
85
86 def update(self):
87 if self.state != PLAYING:
88 return
89
90 # Speed still ramps up, but now stops at the difficulty's ceiling.
91 self.speed = min(self.speed + self.speed_increase_per_frame, self.max_speed)
92 self.player.update()
93
94 self._spawn_timer += 1
95 if self._spawn_timer >= self.spawn_interval:
96 self._spawn_timer = 0
97 self.obstacles.append(Obstacle(self.width, self.ground_y, self.speed))
98
99 for obstacle in self.obstacles:
100 obstacle.move()
main
Ln 100, Col 1 Spaces: 4 UTF-8 CRLF { } Python 3.13.0


File Edit Selection View Go Run Terminal Help
game_engine.py - 39_endless-runner-main - Visual Studio Code
game_engine.py
game > game_engine.py
obstacle.speed = self.speed
Collision uses the swept rect (the whole path travelled this frame),
so a fast obstacle can never skip over the player's hitbox.
player_rect = self.player.rect()
for obstacle in self.obstacles:
if obstacle.swept_rect().colliderect(player_rect):
self.state = GAME_OVER
self.sounds.play("game_over")
return
for obstacle in self.obstacles:
if not obstacle.scored and obstacle.x + obstacle.width < self.player.x:
obstacle.scored = True
self.score += 1
self.sounds.play("score")
self.obstacles = [o for o in self.obstacles if not o.off_screen()]
self.distance += self.speed
def render(self, screen):
pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width, self.ground_y), 4)
pygame.draw.rect(screen, WHITE, self.player.rect())
for obstacle in self.obstacles:
pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())
if self.state == PLAYING:
score_text = self.font.render(f"Score: {self.score}", True, BLACK)
screen.blit(score_text, (10, 10))
level_text = self.small_font.render(self.difficulty, True, BLACK)
screen.blit(level_text, (self.width - level_text.get_width() - 10, 12))
else:
self._render_game_over(screen)
def _centered(self, screen, surface, y):
screen.blit(surface, ((self.width - surface.get_width()) // 2, y))
def _render_game_over(self, screen):
overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
overlay.fill((0, 0, 0, 170))
screen.blit(overlay, (0, 0))
self._centered(screen, self.big_font.render("GAME OVER", True, WHITE), 50)
self._centered(screen, self.font.render(f"Final Score: {self.score}", True, WHITE), 135)
self._centered(screen, self.small_font.render("Play again - choose a difficulty:", True, WHITE), 205)
self._centered(screen, self.small_font.render("1 = Easy 2 = Medium 3 = Hard", True, WHITE), 245)
self._centered(screen, self.small_font.render("Q or Esc = Exit", True, WHITE), 295)
main
Ln 149, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.0

File Edit Selection View Go Run Terminal Help obstacle.py - 39_endless-runner-main - Visual Studio Code

obstacle.py

game > obstacle.py

```python
import math
import pygame


class Obstacle:
    def __init__(self, x, ground_y, speed, width=25, height=40):
        self.x = x
        self.prev_x = x  # where the obstacle was before its latest move
        self.width = width
        self.height = height
        self.y = ground_y - height
        self.speed = speed
        self.scored = False

    def move(self):
        self.prev_x = self.x
        self.x -= self.speed

    def off_screen(self):
        return self.x + self.width < 0

    def rect(self):
        return pygame.Rect(self.x, self.y, self.width, self.height)

    def swept_rect(self):
        """Rectangle covering everything the obstacle passed through this frame.

        rect() only describes where the obstacle ENDED UP. If it moved further
        than the player is wide in one frame, it could jump over the player and
        never overlap. This one stretches from the new position back to the old
        one so no part of the path is skipped.
        """
        left = math.floor(self.x)
        right = math.ceil(self.prev_x + self.width)
        return pygame.Rect(left, self.y, max(right - left, self.width), self.height)
```

main ln 35, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.0

main.py (exit handling)

```python
        engine.render(SCREEN)

        if engine.quit_requested:
            running = False

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()
```

Sounds.py


File Edit Selection View Go Run Terminal Help sound.py - 39_endless-runner-main - Visual Studio Code
sound.py
game > sound.py
import array
import math
import pygame
class SoundManager:
"""Builds the three sound effects in code (no audio files needed).
If the machine has no audio device the game still runs, just silently.
"""
SAMPLE_RATE = 44100
def __init__(self):
self.sounds = {}
self.enabled = False
try:
if not pygame.mixer.get_init():
pygame.mixer.init(self.SAMPLE_RATE, -16, 1, 512)
rate, fmt, channels = pygame.mixer.get_init()
if fmt != -16:
return # unexpected sample format, stay silent rather than crash
self.rate = rate
self.channels = channels
self.sounds["jump"] = self._make([self._tone(300, 600, 0.12, 0.35)])
self.sounds["score"] = self._make([self._tone(880, 1100, 0.10, 0.35)])
self.sounds["game_over"] = self._make([
self._tone(440, 400, 0.18, 0.4),
self._tone(330, 300, 0.18, 0.4),
self._tone(220, 120, 0.40, 0.4),
])
self.enabled = True
except pygame.error:
self.enabled = False
def _tone(self, f_start, f_end, seconds, volume):
"""A sine tone that slides from f_start to f_end and fades out."""
n = int(self.rate * seconds)
out = []
phase = 0.0
for i in range(n):
t = i / n
freq = f_start + (f_end - f_start) * t
phase += 2 * math.pi * freq / self.rate
envelope = 1.0 - t
out.append(int(32767 * volume * envelope * math.sin(phase)))
return out
def _make(self, parts):
samples = [s for part in parts for s in part]
if self.channels == 2:
samples = [s for s in samples for _ in (0, 1)]
return pygame.mixer.Sound(buffer=array.array("h", samples).tobytes())
def play(self, name):
if self.enabled and name in self.sounds:
self.sounds[name].play()
main Ln 58, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.0

PROMPT LOG (SUMMARY):

A short summary of the real conversation, in order. It is not the full transcript; the exported chat is attached separately.

#	What I asked	What the AI did
1	Asked for help with Lab 4 and shared the handout.	AI asked for the assigned repo and listed what I must do myself (videos, push, chat export).
2	Shared my assigned repo link (SETAPESU26/39_endless-runner).	AI read the code, reproduced the collision bug, fixed it, added the game-over screen, replay with difficulty choice, and sound effects, and tested them.
3	Said I forked the repo and asked what to do next.	AI gave steps: run the original, record the 'before' video, copy in the updated code, upload to my fork.
4	Shared a screenshot of a pip error in cmd.	AI explained I had typed two commands together and gave the correct one: python main.py.
5	Shared a screenshot of the game frozen after a hit.	AI explained this is the missing game-over screen (Task 2) and how to record the 'before' video.
6	Asked AI to fix the code and tell me what to do.	AI pointed to the updated code zip and gave steps to replace the files.
7	Asked how to run the zip and how to replace the files.	AI gave step-by-step extract-and-replace instructions.
8	Asked for the changed code, from what to what.	AI made a before/after code document grouped by task.
9	Asked for VS Code-style screenshots of the changed code.	AI generated screenshot images of each changed file.
10	Shared a GitHub error: a file with the same name already exists.	AI explained that existing files must be edited with the pencil icon, not created again.